Benchmark

1. Bevezetés

A benchmark célja nagyméretű, négyzetes mátrixok kiszámítására alkalmas algoritmusok futási idejének mérése, összehasonlítása. A vizsgálat során hagyományos, szekvenciális végrehajtás került összehasonlításra a párhuzamosított implementációkkal. A párhuzamosítás videókártyán történt. Vizsgálatra került az is, hogy milyen futási időket kapunk a videókártya gyorsítótárát kihasználó algoritmussal.

2. A hardver

A benchmarkok egy asztali számítógépen történtek. A komponensek specifikáció:

- **Processzor:** AMD Ryzen 5 3600 (6 mag, 12 szál, 3.6 GHz órajel)
- **Videókártya:** NVIDIA GeForce RTX 4060 (8 GB GDDR6 VRAM)

3. A módszer

A teljesítmény elemzéséhez három eltérő implementáció került vizsgálatra azonos, véletlenszerűen generált lebegőpontos (float) értékekkel feltöltött tesztmátrixokon:

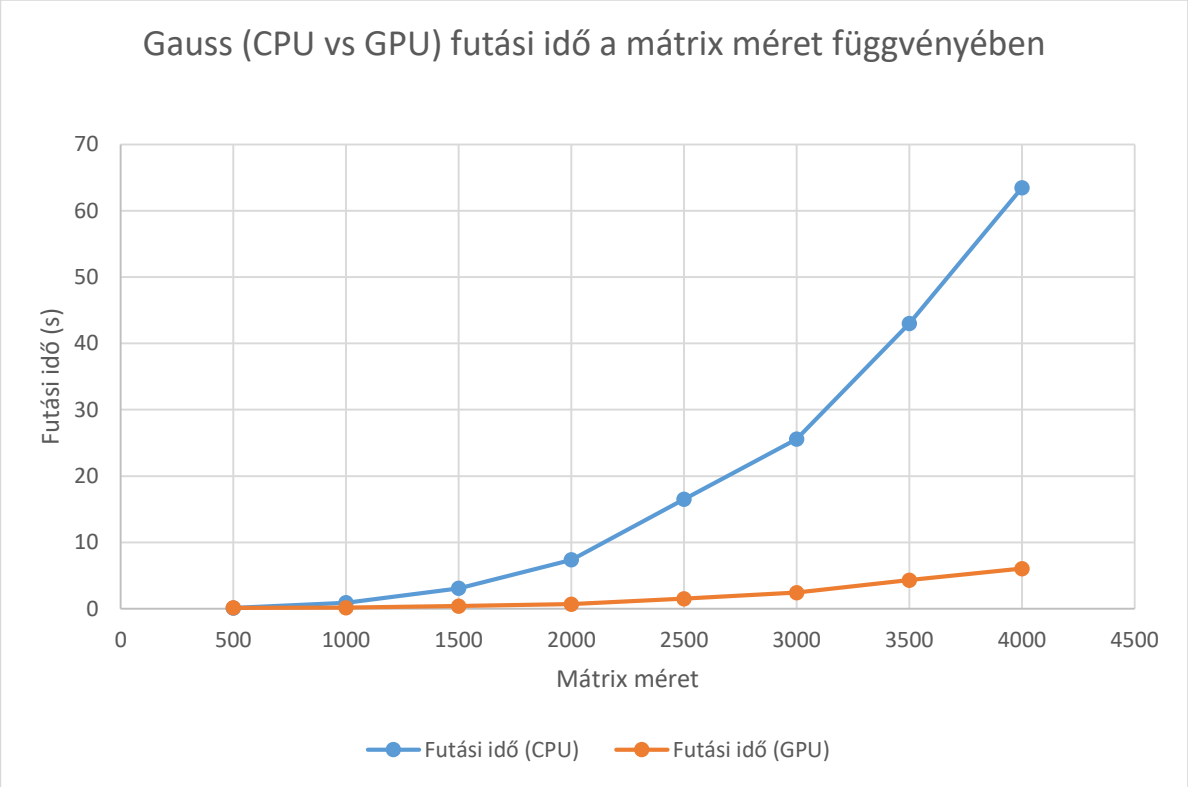
1. **CPU Gauss (szekvenciális):** Hagyományos, egy szálon futó Gauss-elimináció részleges főelem-kiválasztással.
2. **GPU Gauss:** OpenCL alapú megközelítés, amely minden számítási lépésnél közvetlenül a globális videómemóriát (VRAM) olvassa és írja.
3. **GPU LU (Blokkosított):** A mátrixot kisebb, fix méretű csempékre osztja, amelyeket betölt a GPU lokális memóriájába (__local). Ez a módszer minimalizálja a VRAM-kommunikációt.

A szekvenciális CPU implementáció futási idejének mérése a szabványos C könyvtár `clock()` függvényével történt. Ezzel szemben a GPU-s végrehajtási idő mérésére az OpenCL beépített eseménykezelőjét (`CL_PROFILING_COMMAND_START` és `END`) használtuk.

4. Eredmények és kiértékelés

4.1. Szekvenciális és párhuzamos Gauss-eliminációt

Az első mérés során az alapvető CPU-s implementáció került összehasonlításra az OpenCL-ben írt verzióval N = 500 és N = 4000 közötti mátrixméreteken.

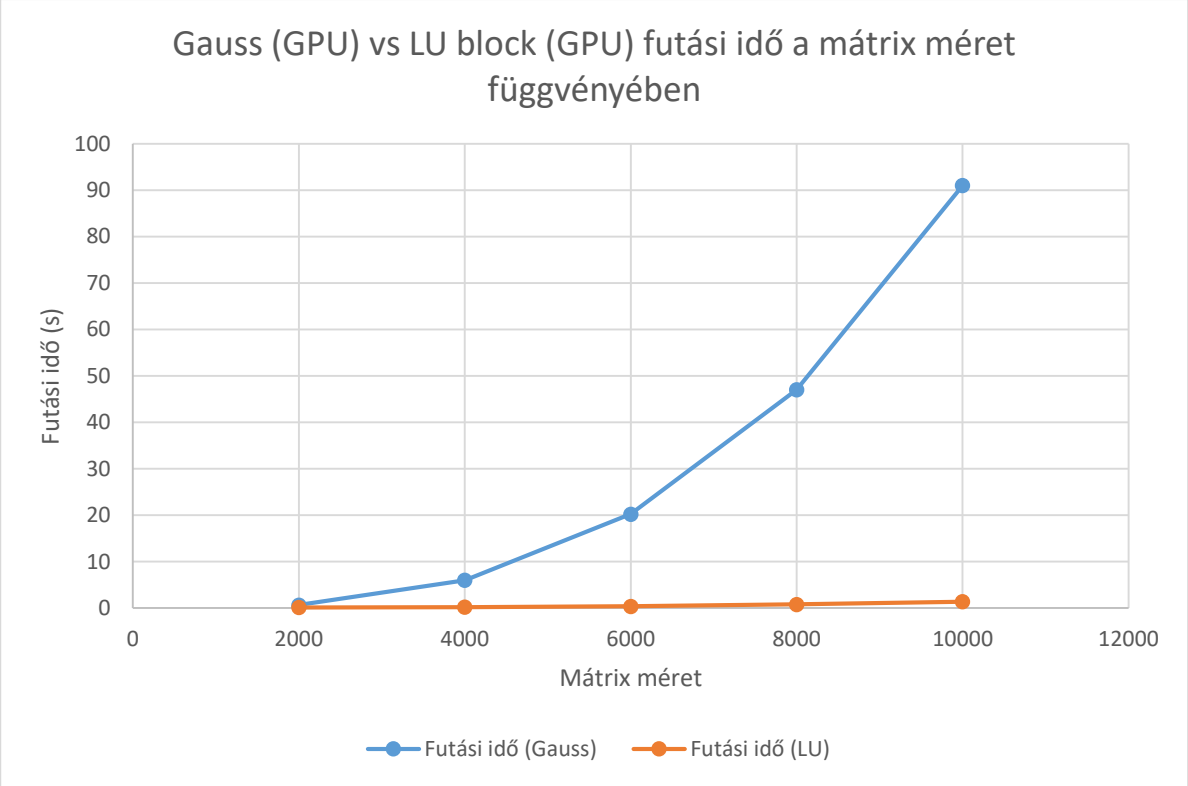


1. ábra

Az *1. ábrán* jól megfigyelhető az algoritmus futási idejének köbös növekedése. Kisebb méreteknél (N = 500) a CPU minimálisan gyorsabb volt, mivel a CPU és GPU közötti kommunikáció ideje túlszárnyalta a párhuzamosításból nyert időt. Azonban az N = 4000 méretnél a GPU már nagyságrendekkel jobban teljesített, több, mint 10-szeres gyorsítás olvasható le.

4.2. Gauss-eliminációés gyorsítótárazott LU-felbontás összehasonlítása GPU-n

A legjelentősebb teljesítménynövekedést az algoritmus blokkosítása adta. A második mérésben a Gauss és a blokkosított LU algoritmusokok kerültek összehasonlításra, kizárólag GPU-n, egészen N = 10000 méretig.

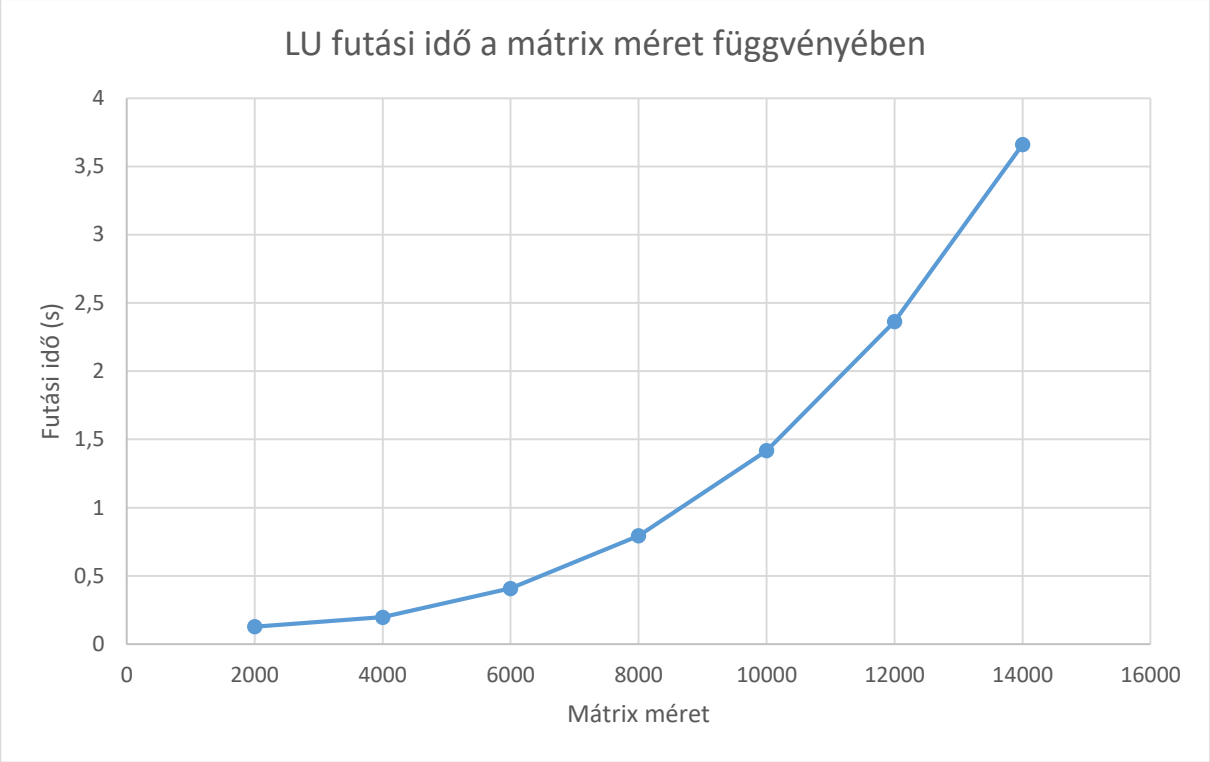


2. ábra

A *2. ábra* látványosan demonstrálja a memóriakezelés fontosságát a párhuzamosítás során. Bár a két algoritmus matematikai műveletigénye megegyezik, az N = 10000 méretű mátrixnál a Gauss 91 másodperc alatt, míg az LU felbontás mindössze 1.3 másodperc alatt végzett. Ez közel 66-szoros gyorsulást jelent ugyanazon a hardveren, amit a lokális memória (__local) használata és a lecsökkentett kernelindítási overhead tett lehetővé.

4.3. A blokkosított LU-felbontás skálázódása extrém mátrixméreteken

Mivel a 2. ábrán a Gauss-eliminació magas futási ideje miatt az LU görbéje látszólag teljesen ellaposodott, szükség volt egy külön mérésre, amely az optimalizált algoritmus viselkedését vizsgálja egy sokkal nagyobb tartományban (N = 2000-től egészen N = 14000-ig).

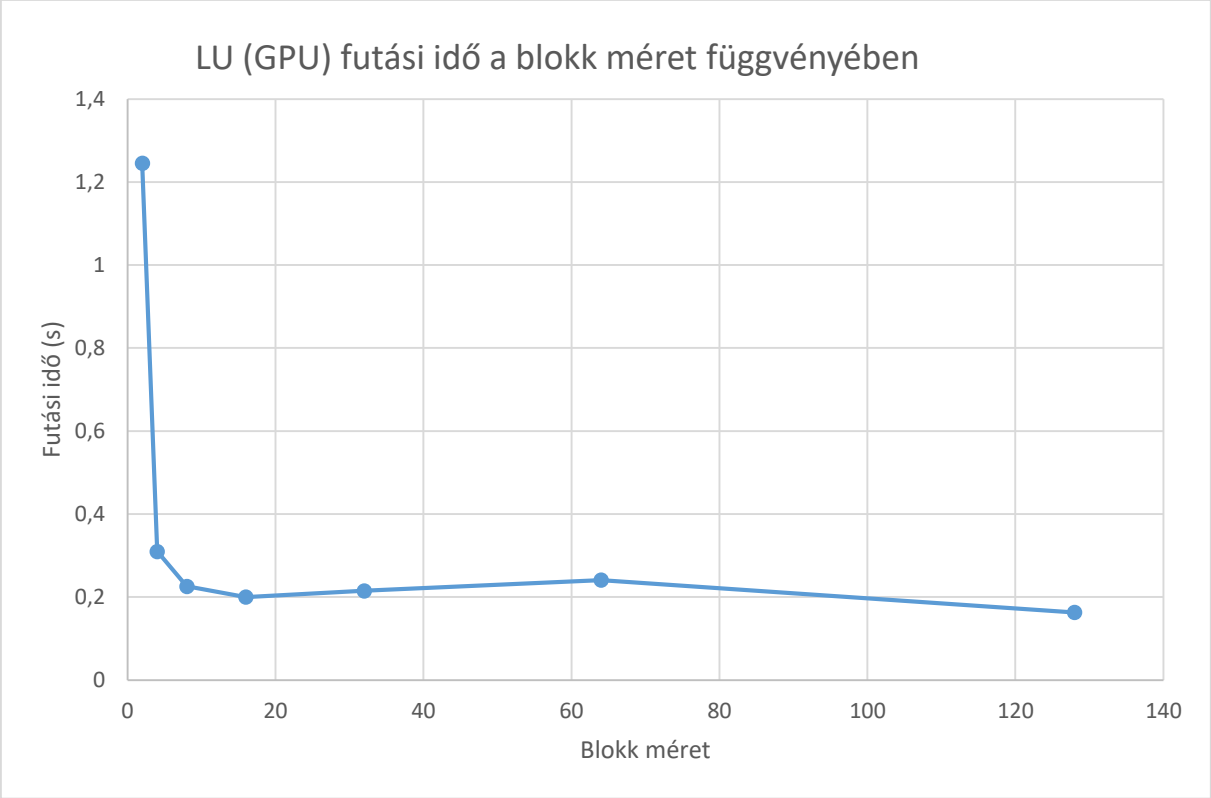


3. ábra

A *3. ábra* rávilágít arra, hogy a hardveres optimalizációk ellenére a probléma komplexitása továbbra is $O(N^3)$ maradt. Azonban a GPU még egy hatalmas, 14000 x 14000-es mátrixszal is képes volt alig több mint 3.6 másodperc alatt végezni.

4.4. A blokkméret finomhangolása az LU algoritmusban

A negyedik mérés a csempézés méretének (BLOCK_SIZE) teljesítményre gyakorolt hatását vizsgálta egy fix, 4096 x 4096-os mátrixon, az LU algoritmus használatával. A BLOCK_SIZE paraméter az eredeti mátrix felbontását határozza meg: definiálja a memóriába beolvasott részmátrixok méretét, illetve így közvetetten a végrehajtandó blokkok darabszámát.



4. ábra

A *4. ábra* görbéjén látható, hogy túl kicsi blokkméretek esetén a futási idő jelentősen megnő. Például egy 2-es blokkméretnél a processzornak nyolcszor annyiszor kell új fázist indítania a GPU-n, mint a 16-os méretnél, ami overheadet okoz. Az optimumot a 16-os méret jelentette. A 32-nél nagyobb értékeknél a kernel munkacsoport mérete már átlépi a hardveres limiteket, ami a teljesítmény stagnálásához, extrém esetben pedig a futás megszakadásához vagy helytelen eredményhez vezet.

5. Összegzés

A mérési eredmények bizonyítják, hogy a nagyméretű mátrixműveleteknél a videókártyák magas számítási kapacitásának kihasználásához messze nem elegendő az algoritmusok egyszerű szálakra bontása. Az igazi áttörést az adatok lokalizációja, a globális memóriahozzáférések minimalizálása és az overhead csökkentése jelentette. A blokkosított LU algoritmus finomhangolásával 66-szoros sebességnövekedés érhető el ugyanazon a videókártyán.